



Enunciado

Em muitos problemas de engenharia e ciências, é necessário encontrar o valor de uma variável que torna uma equação igual a zero. Quando não é possível obter esse valor diretamente por meio de uma fórmula, podem ser usados métodos numéricos iterativos. Esses métodos começam com uma ou mais aproximações iniciais e, a cada iteração, calculam uma nova aproximação para a raiz.

Nesta atividade, você implementará em Python os métodos da **bisseção**, de **Newton** e da **secante**. Em seguida, aplicará os métodos a diferentes funções, analisará os resultados e produzirá animações gráficas que mostrem o processo de aproximação das raízes.

Uma raiz de uma função $f(x)$ é um valor r que satisfaz

$$f(r) = 0.$$

Seu objetivo é encontrar aproximações para as raízes das funções propostas e verificar como o comportamento dos métodos depende da função e das aproximações iniciais escolhidas.

Funções para investigar

Considere as seguintes funções:

$$\begin{aligned} f_1(x) &= x^2 - 2, \\ f_2(x) &= x^3 - x - 2, \\ f_3(x) &= e^{-x} - x, \\ f_4(x) &= x^3 - 2x + 2 \end{aligned}$$

Os valores da tabela são sugestões para iniciar os testes. Você poderá escolher outros valores, desde que justifique suas escolhas. Nas funções trigonométricas, considere x em radianos.

Função	Intervalo para bisseção	Newton: x_0	Secante: (x_0, x_1)
$f_1(x) = x^2 - 2$	$[1, 2]$	1,5	$(1, 2)$
$f_2(x) = x^3 - x - 2$	$[1, 2]$	1,5	$(1, 2)$
$f_3(x) = e^{-x} - x$	$[0, 1]$	0,5	$(0, 1)$
$f_4(x) = x^3 - 2x + 2$	$[-2, -1]$	-2	$(-2, -1)$

Parte 1 – Implementação dos métodos

Implemente cada método em uma função independente no Python. Por exemplo:

- `bissecao(f, a, b, tol, max_iter);`
- `newton(f, df, x0, tol, max_iter);`
- `secante(f, x0, x1, tol, max_iter).`

Cada função deve retornar, no mínimo:

- a aproximação final da raiz;
- o número de iterações realizadas;
- uma lista com as aproximações calculadas em cada iteração;
- uma indicação de convergência ou de falha.

Use uma tolerância de 10^{-6} e um limite máximo de 100 iterações. O código deve detectar situações em que não seja possível continuar, como:

- $f(a)f(b) > 0$ no método da bisseção;
- derivada $f'(x_k)$ igual ou muito próxima de zero no método de Newton;
- denominador igual ou muito próximo de zero no método da secante;
- ausência de convergência dentro do número máximo de iterações.

Para determinar a parada, você pode usar a diferença entre aproximações sucessivas,

$$|x_{k+1} - x_k| < \text{tol},$$

e também verificar o resíduo,

$$|f(x_{k+1})| < \text{tol.}$$

Explique no notebook qual critério foi adotado.

Parte 2 – Aplicação e análise

Organize os resultados em uma tabela com as seguintes colunas:

Função	Método	Valores iniciais	Raiz aproximada	$ f(x) $ final	Iterações/resultado

Parte 3 – Animação gráfica

Crie uma animação para pelo menos três combinações de função e método. A animação deve mostrar:

- o gráfico de $f(x)$ e o eixo horizontal $y = 0$;
- os pontos correspondentes às aproximações produzidas pelo método;
- os segmentos, intervalos ou retas usados para calcular cada nova aproximação;
- o número da iteração;

- o valor atual da aproximação;
- o resíduo $|f(x_k)|$, ou outro indicador de erro escolhido.

Para a bisseção, mostre o intervalo que contém a raiz e sua redução a cada iteração. Para Newton, mostre as retas tangentes à curva. Para a secante, mostre as retas que passam pelos dois pontos usados em cada passo. Exporte cada animação como arquivo GIF e inclua os arquivos no material entregue.

Entregáveis

Cada estudante ou grupo deverá entregar Um dashboard ilustrativo de resultado no <https://www.mathcha.io/> contendo:

1. link para o notebook do Google Colab com as implementações e os testes;
2. os arquivos GIF das animações;
3. a tabela de resultados;

O código deve estar organizado, comentado e permitir que outra pessoa altere a função, os valores iniciais, a tolerância e o número máximo de iterações sem reescrever todo o programa.